# Medical Computer Vision Lab

**Docker vs Kubernetes: Lab Standard for Reproducible ML & Scalable Execution**

Maintainer: Ashery Mbilinyi

Version 1.0 : October 16, 2026

## Contents

**1 Purpose** **2**

**2 Key Definitions** **2**
2.1 What is a Container? . . . 2
2.2 Docker (Containerization) . . . 2
2.3 Kubernetes (Orchestration) . . . 2

**3 Docker vs Kubernetes: Practical Differences** **2**
3.1 Mental Model . . . 2
3.2 Comparison Table . . . 3

**4 Lab Standard (Recommended): Docker + SLURM (Academic Default)** **3**
4.1 Why Docker + SLURM is the Default . . . 3
4.2 Important Note: Docker vs Apptainer/Singularity on HPC . . . 3

**5 When We Use Docker** **3**
5.1 Required (Lab Policy) . . . 3
5.2 Typical Docker Commands . . . 3
5.3 What Belongs in the Docker Image . . . 3

**6 When Kubernetes is Worth It (Optional)** **4**
6.1 Kubernetes Concepts (Minimal Glossary) . . . 4

**7 Recommended Repository Structure (Containers + Experiments)** **4**

**8 Best Practices** **4**
8.1 Reproducibility . . . 4
8.2 Security and Practical Notes . . . 4

**9 Checklist (Code Review)** **5**

**10 Appendix: Minimal Templates** **5**
10.1 Minimal Dockerfile (PyTorch + CUDA) . . . 5
10.2 Minimal SLURM Script (Runs Containerized Training) . . . 5

# 1 Purpose

This document explains the difference between **Docker** and **Kubernetes**, and defines a practical lab standard for using containers in research workflows. The goal is to ensure our ML work is:

- **Reproducible**: the same environment runs across laptops, servers, and cloud
- **Portable**: experiments can move between machines with minimal friction
- **Scalable**: training can be executed reliably on HPC schedulers (e.g., SLURM) and, when needed, on clusters
- **Collaborative**: new lab members can run baselines quickly

# 2 Key Definitions

## 2.1 What is a Container?

A **container** packages:

- code + dependencies (Python libraries, OS packages)
- runtime settings (environment variables, entrypoints)
- a filesystem snapshot (image layers)

This makes training and inference behave consistently across machines.

## 2.2 Docker (Containerization)

**Docker** is primarily a toolchain to **build** and **run** containers.

**What Docker gives us:**

- **Dockerfile** → builds a **Docker image**
- **docker run** → runs the image as a container
- reproducible dependency management (CUDA/PyTorch stacks, system libraries)

## 2.3 Kubernetes (Orchestration)

**Kubernetes (K8s)** is a **container orchestration system**. It schedules and manages containers across a **cluster** of machines.

**What Kubernetes adds on top of containers:**

- automatic scheduling of workloads across many nodes
- scaling (up/down) and self-healing (restart/reschedule)
- service discovery/networking between components
- rolling updates and deployment management

# 3 Docker vs Kubernetes: Practical Differences

## 3.1 Mental Model

- **Docker** = package and run the application reliably
- **Kubernetes** = manage many running containers reliably at cluster scale

### 3.2 Comparison Table

- **Docker** is typically enough for: local dev, single GPU server runs, reproducible environments.
- **Kubernetes** is useful for: cluster-wide scheduling, production deployments, multi-service platforms.

## 4 Lab Standard (Recommended): Docker + SLURM (Academic Default)

In most academic environments, the standard scaling layer is **SLURM**, not Kubernetes.

### 4.1 Why Docker + SLURM is the Default

- Docker gives reproducible environments.
- SLURM schedules jobs on HPC resources (GPUs/CPUs) and handles fair-sharing.
- This is simpler to operate than Kubernetes for typical lab needs.

### 4.2 Important Note: Docker vs Apptainer/Singularity on HPC

Many HPC systems disallow Docker for security reasons. Instead, they use **Apptainer/Singularity** to run containerized workloads.

**Practical pattern:**

```
Docker image  -->  Apptainer/Singularity image  -->  Run via SLURM
```

## 5 When We Use Docker

### 5.1 Required (Lab Policy)

All ML repositories should provide a container recipe:

- `Dockerfile` (preferred), or
- Apptainer recipe / documented conversion steps (if HPC requires it)

### 5.2 Typical Docker Commands

```
# Build
docker build -t mcv-project:latest .

# Run (GPU)
docker run --gpus all -it --rm \
  -v $(pwd):/workspace \
  mcv-project:latest
```

### 5.3 What Belongs in the Docker Image

- pinned Python dependencies (requirements.txt / poetry.lock / conda env)
- system libs needed for the project (e.g., OpenCV deps)
- fixed CUDA/PyTorch compatibility (base image choice matters)

# 6 When Kubernetes is Worth It (Optional)

Kubernetes is not required for most lab research workflows. It becomes valuable if we run a shared platform, for example:

- multiple groups share a GPU cluster and need a common scheduling layer
- we host persistent services (APIs, dashboards, model serving endpoints)
- we run many pipelines continuously (training + evaluation + deployment)

## 6.1 Kubernetes Concepts (Minimal Glossary)

- **Pod**: smallest deployable unit (one or more containers)
- **Deployment**: manages a replicated set of pods (rolling updates)
- **Job**: runs a pod to completion (batch workloads)
- **Service**: stable network endpoint for accessing pods
- **Ingress**: routes external traffic to services

# 7 Recommended Repository Structure (Containers + Experiments)

All projects should follow a consistent structure:

```
project/
  Dockerfile
  requirements.txt (or environment.yml)
  src/
  scripts/
    train.py
    eval.py
  configs/
    baseline.yaml
  slurm/
    train.slurm
  runs/               # outputs (gitignored)
  README.md
```

# 8 Best Practices

## 8.1 Reproducibility

- Pin dependencies (exact versions).
- Save runtime config per run (`config.json/yaml` + `cmd.txt`).
- Save checkpoints (best + last) and log metrics.

## 8.2 Security and Practical Notes

- Do not bake secrets (API keys) into images.
- Prefer environment variables or secret managers.
- Use read-only mounts where possible when evaluating.

# 9 Checklist (Code Review)

Before merging a new training pipeline:

- **Container exists**: repo includes `Dockerfile` (or Apptainer recipe).
- **One-command run**: README has build + run instructions.
- **Reproducible outputs**: runs save config + command + checkpoints.
- **HPC compatibility**: provide `slurm/train.slurm` if used on cluster.

# 10 Appendix: Minimal Templates

## 10.1 Minimal Dockerfile (PyTorch + CUDA)

```
FROM pytorch/pytorch:2.3.1-cuda12.1-cudnn8-runtime

WORKDIR /workspace

# System deps (adjust as needed)
RUN apt-get update && apt-get install -y --no-install-recommends \
    git curl ca-certificates \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /workspace/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY . /workspace

# Default command (override in docker run if needed)
CMD ["python", "scripts/train.py", "--help"]
```

## 10.2 Minimal SLURM Script (Runs Containerized Training)

```
#!/bin/bash
#SBATCH --job-name=mcv_train
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=12:00:00
#SBATCH --output=slurm-%j.out

module load apptainer

# Example: run an Apptainer image built from Docker
apptainer exec --nv mcv-project.sif \
  python scripts/train.py --config configs/baseline.yaml
```